UNIVERSIDAD TECNOLÓGICA NACIONAL

Trabajo Final Integrador
Base de Datos II

Alumnos:
Aldana, Lara Agustina - Comisión 4
Miglioranza, Manuel Agustín - Comisión 3
Rodriguez, Florencia Mabel - Comisión 8

Coordinador
Bruselario, Sebastián

Profesores
Di Mónaco, Florencia
Fonzo, Santiago

Tutores
Baptista Carvalho, Gabriela
Argenti, Bruno
Gubiotti, Florencia Camila

30 de Octubre de 2026

Tabla de contenido

1. Modelado y Definición de constraints …… **3** pág.
2. Integración, Operaciones CRUD…… **8** pág.
3. Mecanismos de Resguardo (Backups)…… **16** pág.
4. Conclusión ……..…….…. **18** pág.

1. Modelado y definición de constraints

Se modela una tienda de comida que recibe pedidos. Tiene por requisito registrar usuarios, organizar productos por categoría (pizzas, bebidas, etc), controlar el stock y guardar cada pedido con su detalle (qué productos, cuántos y a qué precio).

Se modela una base de datos en MongoDB y una aplicación simple a los efectos de que los dueños de la tienda puedan consultar y registrar las operaciones más comunes y poder sacar conclusiones de su negocio que le permitan una mejor gestión actual y futura: saber qué se vendió, a quién, cuándo, cuáles son los estados de los pedidos actuales. Esto permite un mejor control y trazabilidad del giro y la operación cotidiana de la tienda.

MongoDB resulta la elección adecuada para este caso porque en esta instancia el negocio recién está comenzando y necesitan una solución flexible sobre la cual puedan trabajar (no están seguros de la estructura de la información que deben almacenar en lo que respecta a cada entidad o documento, y asumen de antemano que la misma puede cambiar debido a la variedad de productos que venden y que planean incorporar).

Al mismo tiempo, MongoDB permite almacenar de forma eficiente información relacionada, principalmente con embeddings, para evitar consultas tipo JOIN. Un claro ejemplo de esto es los pedidos y sus respectivos detalles. La tienda espera tener una clientela de rápido crecimiento y no quieren que el registro y consulta de operaciones se convierta en un cuello de botella en el corto plazo, por lo que la escalabilidad al elegir MongoDB se encontraría resuelta visto que esta base de datos está diseñada con ello en mente.

Se definieron las siguientes colecciones:

1) **Usuarios**

   Esta colección se utiliza principalmente para clientes y administradores (quienes operan la base de datos y aplicación).

   Es referenciada desde pedidos (cada pedido tiene un usuario asociado obligatoriamente, no hay pedido sin usuario).

   Como datos embebidos se agrega la dirección, al ser esta una información que se consulta de forma conjunta (se busca conocer la dirección de un usuario para saber a donde debe ser entregado el pedido). La relación entre usuario y dirección ies 1:1, siempre se leen juntas

   Ejemplo de documento:

```
{
 "_id": "ObjectId('665f1c2e8a1b2c3d4e5f6b01')",
 "nombre": "Lucía",
 "apellido": "Fernández",
 "email": "lucia@example.com",
 "telefono": "+54 11 5555-1234",
 "rol": "cliente",
 "direccion": {
   "calle": "Av. Santa Fe 1234",
   "localidad": "Pilar",
```

```
    "provincia": "Buenos Aires"
  },
  "activo": true,
  "fecha_alta": "ISODate('2026-10-26T15:00:00Z')",
  "fecha_baja": null
}
```

2) Categorías

Con esta colección se permite agrupar productos y ayuda a entender a la tienda qué tipo de productos son más solicitados o vendidos y cuáles menos. Tiene un ciclo de vida distinto y separado de los productos (ej: un día la tienda decide dejar de vender “pastas”, dar de baja lógica ese todos los productos es más eficiente que tener que escanear todos los documentos con la cadena “pastas” en la propiedad “categoría”)

Es referenciada desde productos mediante su id (una categoría puede ser asignada a muchos productos pero un producto tiene una única categoría: esto, de todas formas, puede modificarse en el futuro – productos con más de una categoría – , motivo por el cual la elección de MongoDB es apropiada para mantener la flexibilidad en la determinación de las relaciones entre colecciones). Renombrar o modificar una categoría se realiza en un único lugar sin tener que modificar ni recorrer los productos que la referencia. Estos son los motivos por los que se decisión por el linking en vez de embeber la categoría en los productos.

Ejemplo de documento:

```
{
  "_id": "ObjectId('665f1c2e8a1b2c3d4e5f6b02')",
  "nombre": "Pizzas",
  "descripcion": "Pizzas artesanales",
  "activo": true,
  "fecha_alta": "ISODate('2026-10-26T15:01:00Z')",
  "fecha_baja": null
}
```

3) Productos

La colección principal sobre la cual se puede controlar qué se vende, a qué precio y cuál es el stock restante. Se relaciona con categorías mediante linking, es decir, referenciando el documento correspondiente en aquella colección mediante su id.

Ejemplo de documento:

```
{
  "_id": "ObjectId('665f1c2e8a1b2c3d4e5f6b03')",
  "nombre": "Pizza Muzzarella",
  "precio": 8500,
  "stock": 20,
  "categoria_id": "ObjectId('665f1c2e8a1b2c3d4e5f6b02')",
  "activo": true,
  "fecha_alta": "ISODate('2026-10-26T15:02:00Z')",
  "fecha_baja": null
}
```

4) Pedidos

Esta colección registra las compras realizadas por los clientes. Se relacionan con los usuarios referenciando el id de estos últimos, embeben la dirección de entrega de forma parcial para fácil acceso y contienen de forma embebida los detalles del pedido, utilizando el subtotal de cada uno de estos para calcular el total del pedido. Los detalles de pedido a su vez hacen referencia a los productos mediante su id, pero copian el precio unitario y el nombre de cada producto referenciado para tener información mínima de consulta relevante al pedido (saber qué se pidió, cual era el precio al momento de crearse el pedido, cuántos productos se pidieron y cuál era el subtotal).

Se consideró apropiado utilizar embedding para los detalles por lo siguiente: Un detalle no tiene sentido sin su pedido, se lee siempre con él y tiene pocas líneas (acotado). Se guarda de forma atómica junto al pedido y evita una cuarta colección con JOINs ($lookup). Además guarda un resumen de información sobre nombre y precio: si el producto cambia de precio mañana, el pedido histórico no se altera.

Sobre la relación de linking para usuario mediante usuario_id, entendemos que ambas entidades tienen ciclos de vida separado, se consultan y editan de forma separada e independiente, por lo que relacionarlos como embedding sería ineficiente y trabajoso para mantener, bastando simplemente que los pedidos referencien a su usuario por id.

Sobre los detalles referenciando los productos por su id mediante linking, similar justificación aplica visto los ciclos de vida separados, entendiendo que los detalles registran los productos de forma histórica (al momento de realizar el pedido), pudiendo los productos modificarse ulteriormente pero siendo ello inaplicable para el detalle: el cliente que realizó un pedido de un producto eligió un precio para comprarlo, por lo que ese precio debe respetarse y no puede actualizarse luego de que el pedido se haya creado, siendo esta una regla de negocio obligatoria para la tienda de comida.

Ejemplo de documento:

```
{
 "_id": "ObjectId('665f1c2e8a1b2c3d4e5f6b04')",
 "usuario_id": "ObjectId('665f1c2e8a1b2c3d4e5f6b01')",
 "fecha_pedido": "ISODate('2026-10-26T20:15:00Z')",
 "estado": "pendiente",
 "direccion_entrega": {
   "calle": "Av. Santa Fe 1234",
   "localidad": "Pilar",
   "provincia": "Buenos Aires"
 },
 "detalle": [
   {
     "producto_id": "ObjectId('665f1c2e8a1b2c3d4e5f6b03')",
     "nombre": "Pizza Muzzarella",
     "precio_unitario": 8500,
     "cantidad": 2,
     "subtotal": 17000
   },
   {
```

```
      "producto_id": "ObjectId('665f1c2e8a1b2c3d4e5f6b05')",
      "nombre": "Fugazzeta",
      "precio_unitario": 9800,
      "cantidad": 1,
      "subtotal": 9800
    }
  ],
  "total": 26800,
  "activo": true,
  "fecha_alta": "ISODate('2026-10-26T20:15:00Z')",
  "fecha_baja": null
}
```

Resumen de relaciones entre colecciones:

Para definir las relaciones se utilizó el criterio “data that is accessed together, should be stored together”. Es decir, si dos conjuntos de datos suelen consultarse o modificarse juntos, deben almacenarse juntos, generalmente mediante documentos embebidos. En cambio, si una relación requiere independencia, tamaño variable o acceso frecuente por separado, se opta por referencias entre colecciones, todo ello procurando equilibrar la redundancia de los datos almacenados con el rendimiento de las consultas y operaciones de escritura.

Relación	Cardinalidad	Campo que la implementa	Tipo	Cómo se lee	Efecto de la baja lógica
categorias y productos	1 : N	productos.categoria_id	Linking	$lookup	Al dar de baja una categoría, se dan de baja sus productos
usuarios y pedidos	1 : N	pedidos.usuario_id	Linking	$lookup	Al dar de baja un usuario, se dan de baja sus pedidos pendientes y en preparación; los entregados quedan como historial
pedidos y detalle	1 : N	pedidos.detalle[]	Embedding	Viene en el mismo documento	Se da de baja junto con el pedido
productos y líneas del detalle	1 : N	pedidos.detalle[].producto_id	Linking, más info de nombre y precio	$lookup solo si se necesita el producto vigente	Dar de baja un producto no altera los pedidos ya hechos
usuarios y dirección	1 : 1	usuarios.direccion	Embedding	Viene en el mismo	Se da de baja junto con el usuario

				documento	
pedidos y dirección de entrega	1 : 1	pedidos.direccion_ entrega	Embedding (copia de la dirección del usuario)	Viene en el mismo documento	Se da de baja junto con el pedido

Una nota sobre la **baja lógica** aplicada a la arquitectura diseñada: todos los documentos que se ingresan en la base de datos permanecen en ella y sólo se “eliminan” utilizando el valor booleano de “activo” y registrando la “fecha_baja”. Esto permite una mejor auditoria respecto de la información de la base, permitiendo reconstruir y entender la evolución de la misma en retrospectiva ya que no faltará ningún documento en la misma y evitará tener relaciones rotas entre colecciones (en particular aquellas que se realizan mediante linking referenciando documentos mediante id).

2. Integración, operaciones CRUD

Se desarrolló una aplicación simple en javascript utilizando Node, remitimos al código en la carpeta **src** del proyecto y al README para la instalación de dependencias y las instrucciones para levantar e interactuar con la aplicación y con la base de datos que debe crearse en MongoDB Atlas. Se optó por la división en módulos para mejor organización y menor repetición de código.

Para simplificar el diseño del funcionamiento de la aplicación, los módulos funcionan como scripts que se ejecutan desde una terminal utilizando el driver nativo de MongoDB. La arquitectura cliente-servidor se utiliza en tanto y en cuanto la aplicación interactúa con el driver de mongodb que proporciona el cliente de la base de datos, pero no en el sentido de un servidor web que mantiene una conexión abierta esperando requests en determinados endpoints para redireccionarlos al cliente de la base de datos.

Es posible verificar las operaciones realizadas en la base de datos desde una terminal (en caso de que se desee correr la aplicación localmente) o bien desde el GUI de Mongo Atlas o utilizando Mongo Compass u otro gestor de base de datos (NoSQL Booster for MongoDB, etc). Lo primordial en cualquier caso es proveer de forma correcta la URI de conexión a la base de datos, indicada en el archivo de entorno **.env** en la variable **MONGODB_URI**.

Los módulos de la aplicación se dividen en 4:

db.js: Este módulo se encarga de levantar un cliente de mongo con el paquete “mongodb”, conectándose a un cluster de MongoDB Atlas mediante la URI. Tiene a tarea de conectarse y devolver la conexión de la base de datos y de cerrar la conexión.

setup.js: Contiene la estructura base y validadores para los distintos esquemas de las 4 colecciones (usuarios, categorías, productos y pedidos) y utiliza la API expuesta por **db.js** para conectarse a la base de datos, inicializarla y crear índices de forma asíncrona utilizando la API de Promises de javascript (ES6+). Está diseñado para crear los esquemas y validaciones una vez y no sobreescribir los mismos en caso de que ya existan en la base de datos (si se corrió la lógica de forma previa). Cierra la conexión si no hay fallos o arroja una promesa rechazada en caso de que haya algún error.

crud.js: Es el módulo encargado de exponer la API de CRUD para las 4 colecciones, validando las colecciones (no permitiendo crear nuevas). Maneja de forma específica aquellas colecciones que requieren que existan documentos relacionados de forma previa a su inserción (crearPedido verifica que existan usuarios y productos y no se inserten pedidos sin usuarios o sin productos) y de ejecutar la lógica de negocio necesaria (bajar stock de producto por la cantidad que sea indicada para el detalle del pedido y denegar la inserción de un pedido que indique en su detalle mayor stock que el disponible en el producto que indica). Contiene operaciones de lectura que permite hacer consultas que relacionan colecciones usando el operador $lookup (listarProductosConCategoria) y se encarga de gestionar la baja lógica de los documentos, propagando sus efectos para los casos de categorías (que propaga sobre productos de la misma categoría) y usuarios (que propaga sobre pedidos de un mismo usuario que se encuentren pendientes o en preparación).

Sobre cómo se maneja la baja lógica de los documentos, aquellos documentos dados de baja lógica (activo: false) se consideran "eliminados" por lo que no corresponde su modificación posterior en ninguno de los campos posteriores a la baja lógica, a modo de preservar el último estado del documento previo a dicha baja y permitir una reconstrucción fidedigna del “pasado” de la base de datos, asegurando consistencia lógica, trazabilidad y auditoría de la base de datos.

demo.js: Es un script que ejecuta a forma de ejemplo el funcionamiento de la base de datos y la lógica desarrollada en los otros módulos de la aplicación.

Se destacan las funciones principales:

Conexión a la base de datos:

```
const client = new MongoClient(uri, {
serverSelectionTimeoutMS: 10000 });
async function getDb() {
 await client.connect();
 return client.db(process.env.DB_NAME || "tienda_comida");
}
```

CREATE:

```
async function crear(db, col, datos) {
 validarColeccion(col);
 const doc = {
   ...datos,
   activo: true,
   fecha_alta: new Date(),
   fecha_baja: null,
 };
 const r = await db.collection(col).insertOne(doc);
 return { _id: r.insertedId, ...doc };
}
```

READ (no lista documentos “eliminados”):

```
async function listar(db, col, filtro = {}) {
 validarColeccion(col);
 return db
   .collection(col)
   .find({ ...filtro, activo: true })
   .toArray();
}
```

UPDATE (sólo sobre documentos no “eliminados”):

```
async function actualizar(db, col, id, campos) {
 validarColeccion(col);
 const datos = { ...campos };
 ["_id", "activo", "fecha_alta", "fecha_baja"].forEach((k) =>
delete datos[k]);
 const r = await db
   .collection(col)
   .updateOne(
     { _id: oid(id), activo: true },
```

```
    { $set: { ...datos, fecha_modificacion: new Date() } },
  );
 return { encontrados: r.matchedCount, modificados:
r.modifiedCount };
}
```

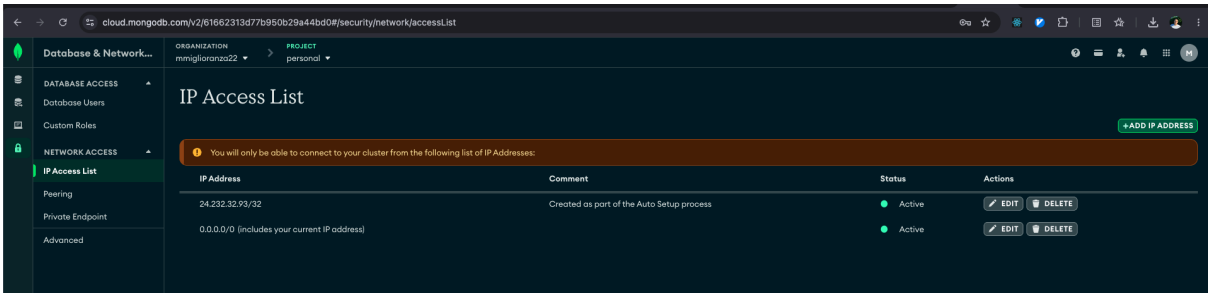
DELETE (baja lógica):

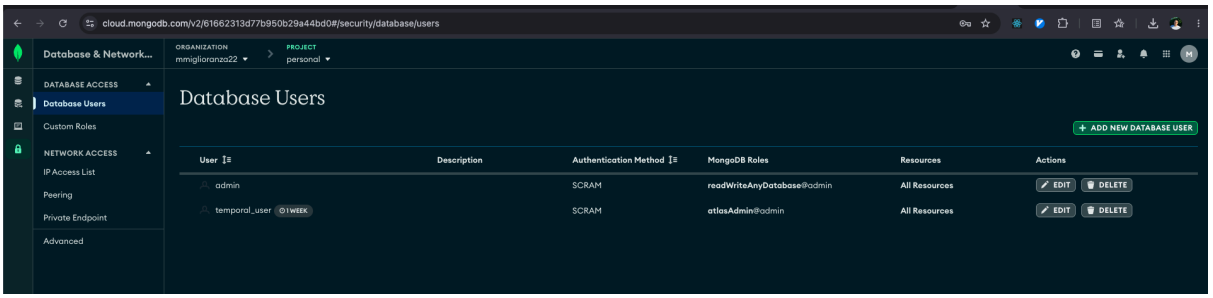
```
async function bajaLogica(db, col, id) {
 validarColeccion(col);
 const _id = oid(id);
 const baja = { $set: { activo: false, fecha_baja: new Date()
} };
 const r = await db.collection(col).updateOne({ _id, activo:
true }, baja);

 // Cascada (también lógica). Los pedidos ya entregados se
conservan como historial.
 if (r.modifiedCount === 1) {
   if (col === "categorias") {
     await db
       .collection("productos")
       .updateMany({ categoria_id: _id, activo: true }, baja);
   }
   if (col === "usuarios") {
     await db.collection("pedidos").updateMany(
       {
         usuario_id: _id,
         activo: true,
         estado: { $in: ["pendiente", "preparando"] },
       },
       baja,
     );
   }
 }
 return { encontrados: r.matchedCount, dadosDeBaja:
r.modifiedCount };
}
```

Sobre la conexión al cluster gratuito de MongoDB Atlas, como bien se encuentra indicado en el README.md, se provee un usuario temporal para que se pueda hacer un testeo más sencillo de la aplicación sin necesidad de configurar manualmente el MONGODB_URI. (La contraseña es la misma que el nombre del usuario temporal).

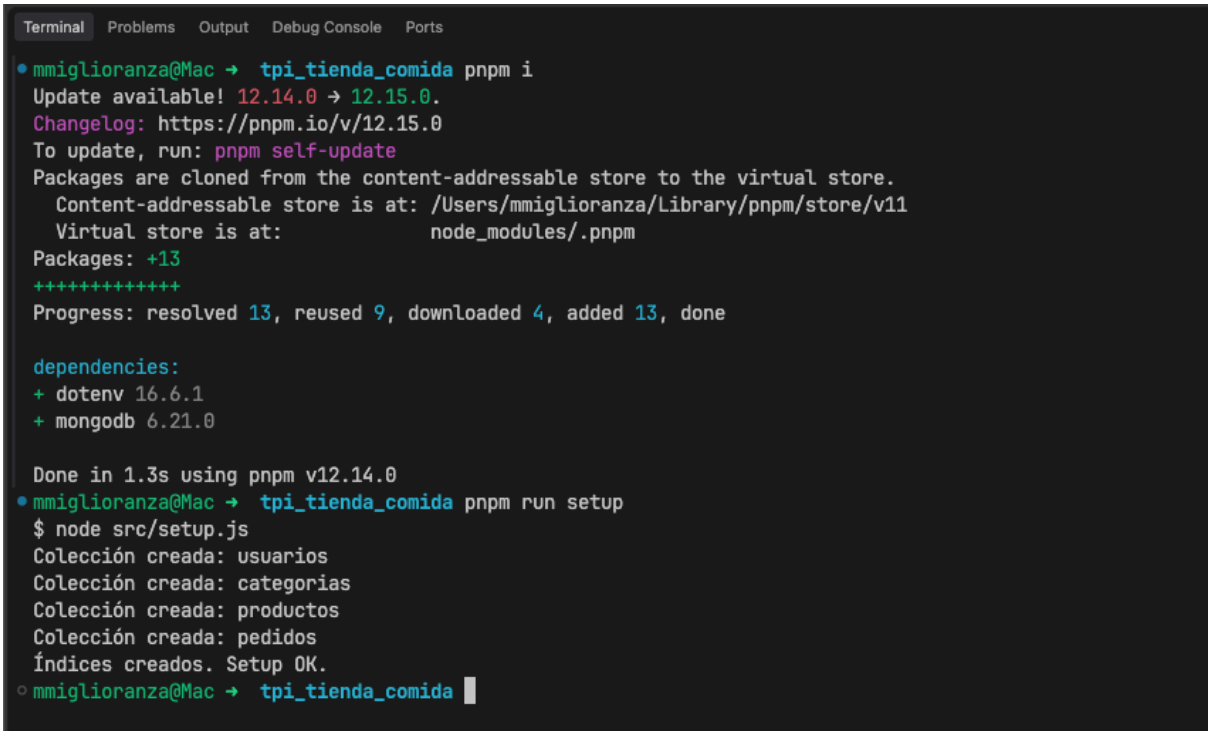
Se adjuntan las capturas del Database Access y Network Access:



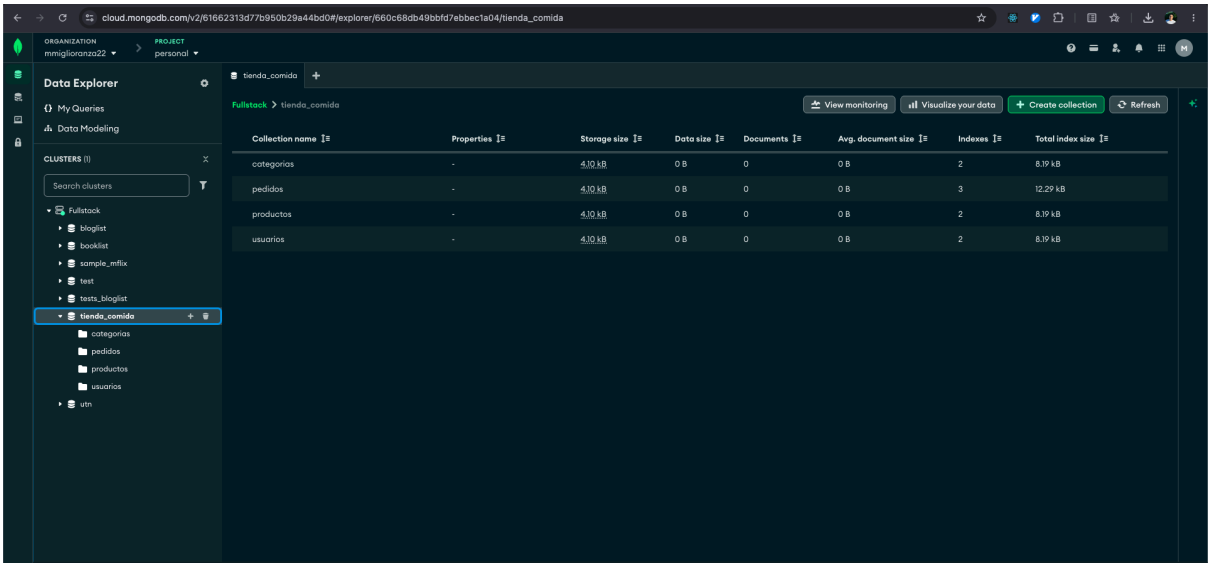


Se adjuntan capturas del funcionamiento de la aplicación en un entorno cloud (MongoDB Atlas):

Instalación de dependencias e inicialización de base de datos (setup.js)



Vista desde Mongo Atlas (colecciones vacías):



Creación de documentos de prueba (demo.js)

```
mmiglioranza@Mac → tpi_tienda_comida pnpm run demo
$ node src/demo.js
Conectado a MongoDB Atlas, base: tienda_comida

=== CREATE ===
Pedido creado: 6ae0ef32c384c22d858909bc | total: 26800

=== READ (solo activos, con $lookup) ===
```

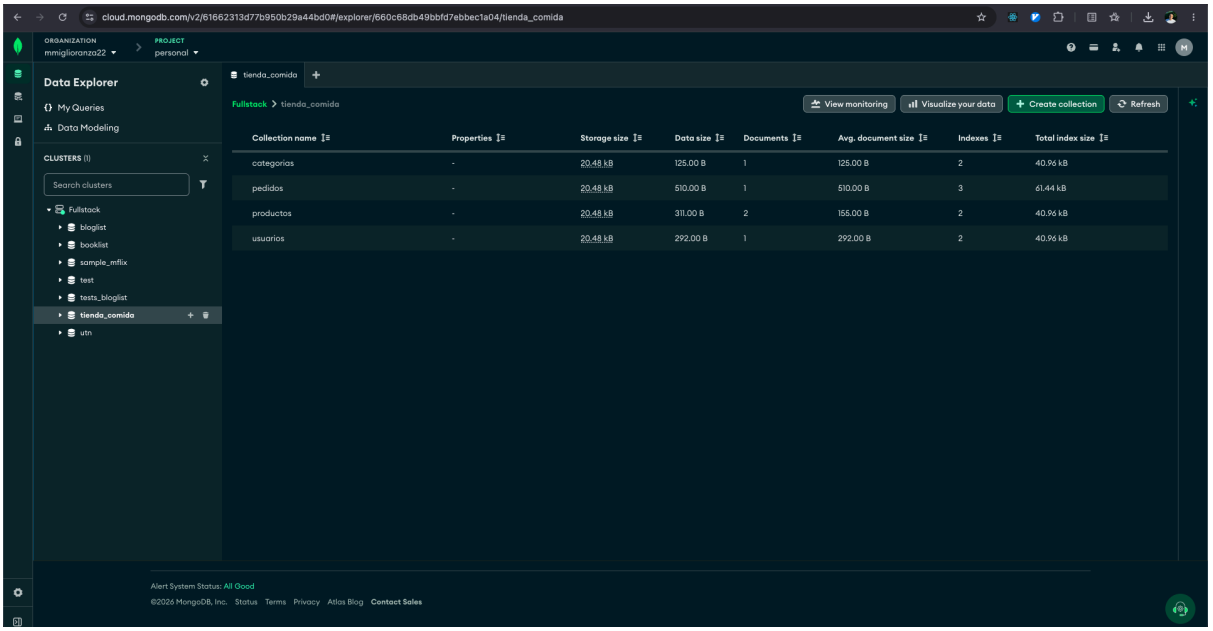
(index)	_id	nombre	precio	stock	categoria
0	new ObjectId('6ae0ef32c384c22d858909ba')	'Fugazzeta'	9800	14	'Pizzas 698298'
1	new ObjectId('6ae0ef32c384c22d858909b9')	'Pizza Muzzarella'	8500	18	'Pizzas 698298'

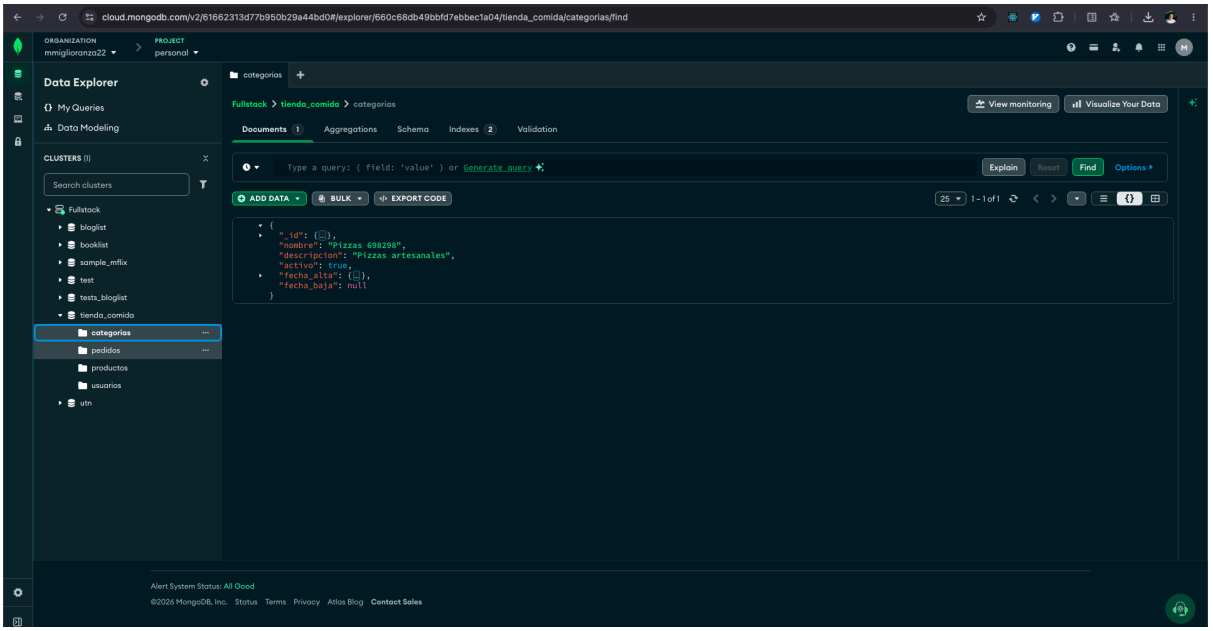
(index)	usuario	items	total	estado
0	'Lucía Fernández'	2	26800	'pendiente'

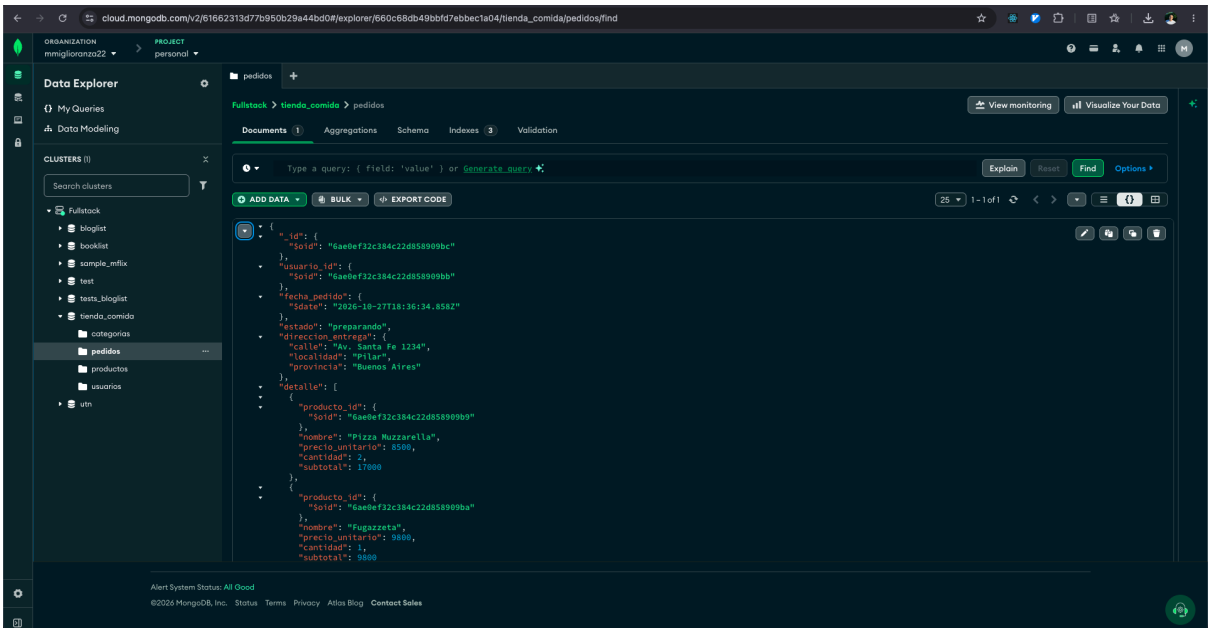
```
=== UPDATE ===
Precio: { encontrados: 1, modificados: 1 }
Estado: { encontrados: 1, modificados: 1 }

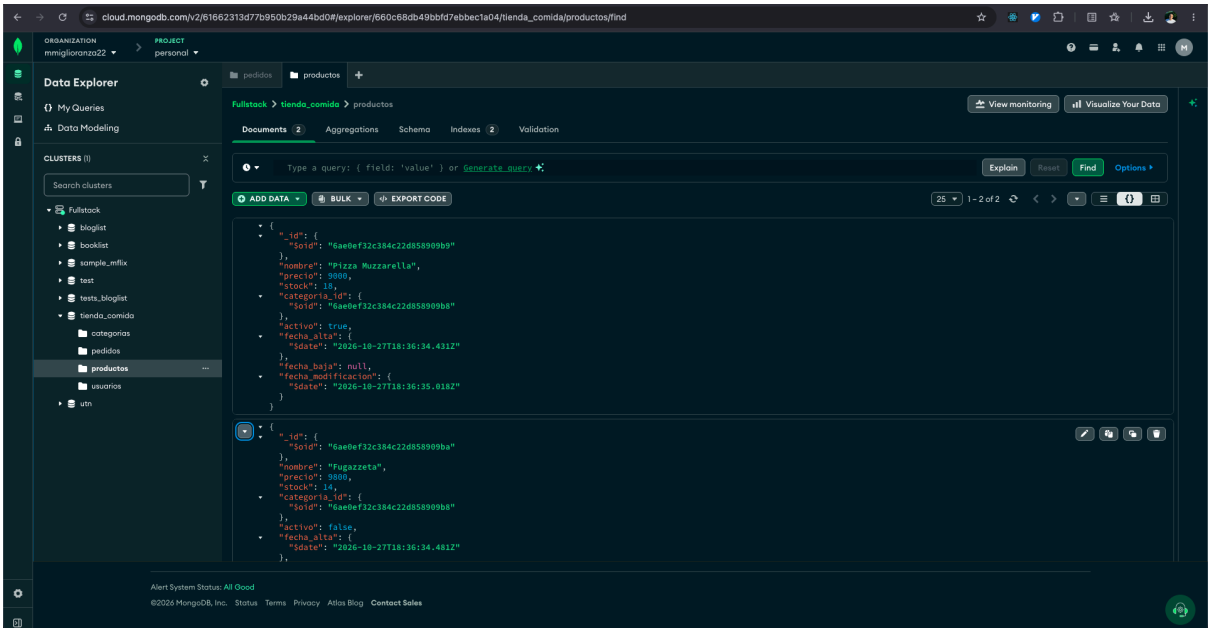
=== DELETE (baja lógica) ===
{ encontrados: 1, dadosDeBaja: 1 }
```

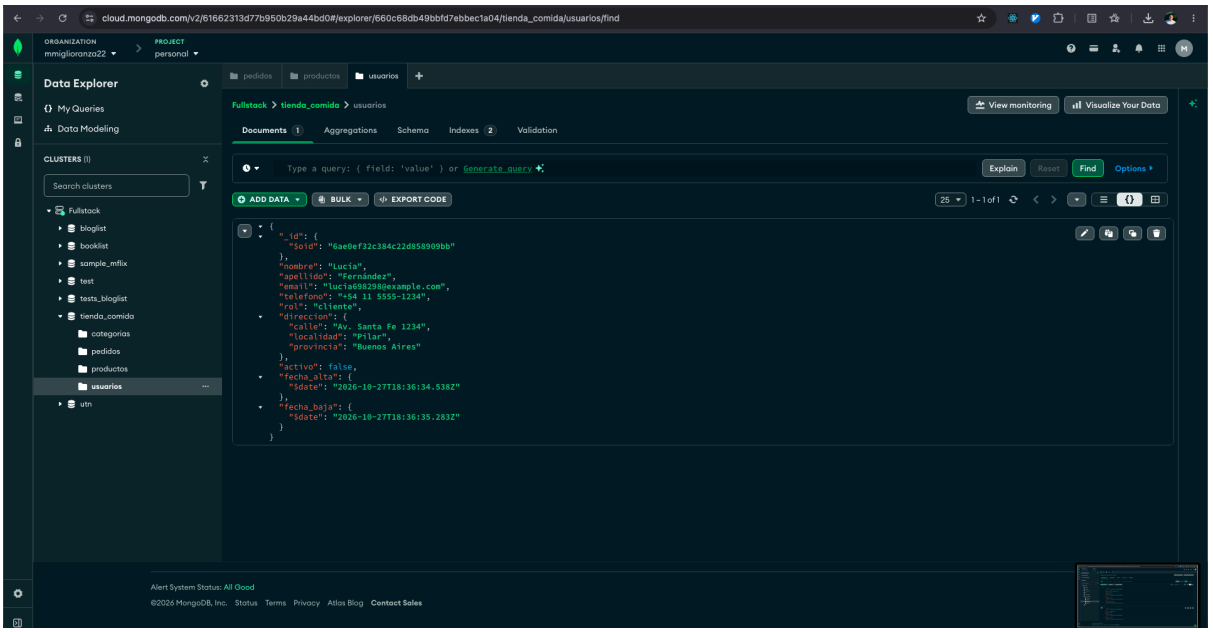
Vista de colecciones en Mongo Atlas:



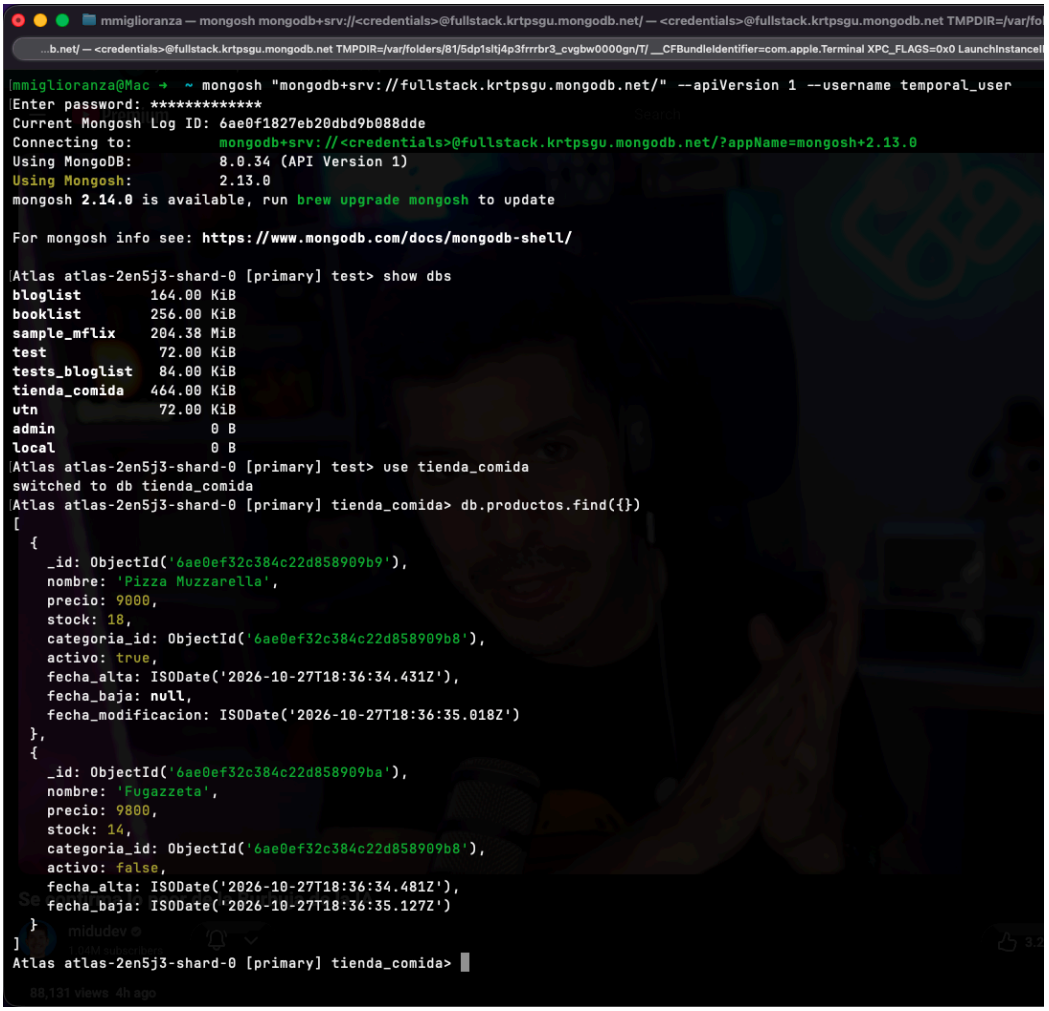



cloud.mongodb.com/v2/61662313d77b950b29a44bd0#/explorer/660c68db49bbfd7ebbec1a04/tienda_comida/pedidos/find
ORGANIZATION
mmiglioranza22
PROJECT
personal
Data Explorer
My Queries
Data Modeling
CLUSTERS (1)
Search clusters
Fullstack
bloglist
booklist
sample_mflix
test
tests_bloglist
tienda_comida
categorias
pedidos
productos
usuarios
utn
pedidos
Fullstack > tienda_comida > pedidos
View monitoring
Visualize Your Data
Documents 1
Aggregations
Schema
Indexes 3
Validation
Type a query: { field: 'value' } or Generate query
Explain
Reset
Find
Options
ADD DATA
BULK
EXPORT CODE
25
1 - 1 of 1
{
"_id": {
"$oid": "6ae0ef32c384c22d858909bc"
},
"usuario_id": {
"$oid": "6ae0ef32c384c22d858909bb"
},
"fecha_pedido": {
"$date": "2026-10-27T18:36:34.858Z"
},
"estado": "preparando",
"direccion_entrega": {
"calle": "Av. Santa Fe 1234",
"localidad": "Pilar",
"provincia": "Buenos Aires"
},
"detalle": [
{
"producto_id": {
"$oid": "6ae0ef32c384c22d858909b9"
},
"nombre": "Pizza Muzzarella",
"precio_unitario": 8500,
"cantidad": 2,
"subtotal": 17000
},
{
"producto_id": {
"$oid": "6ae0ef32c384c22d858909ba"
},
"nombre": "Fugazzeta",
"precio_unitario": 9800,
"cantidad": 1,
"subtotal": 9800
Alert System Status: All Good
©2026 MongoDB, Inc. Status Terms Privacy Atlas Blog Contact Sales


cloud.mongodb.com/v2/61662313d77b950b29a44bd0#/explorer/660c68db49bbfd7ebbec1a04/tienda_comida/productos/find
ORGANIZATION
mmiglioranza22
PROJECT
personal
Data Explorer
My Queries
Data Modeling
CLUSTERS (1)
Search clusters
Fullstack
bloglist
booklist
sample_mflix
test
tests_bloglist
tienda_comida
categorias
pedidos
productos
usuarios
utn
pedidos
productos
Fullstack > tienda_comida > productos
View monitoring
Visualize Your Data
Documents 2
Aggregations
Schema
Indexes 2
Validation
Type a query: { field: 'value' } or Generate query
Explain
Reset
Find
Options
ADD DATA
BULK
EXPORT CODE
25
1 - 2 of 2
{
"_id": {
"$oid": "6ae0ef32c384c22d858909b9"
},
"nombre": "Pizza Muzzarella",
"precio": 9800,
"stock": 18,
"categoria_id": {
"$oid": "6ae0ef32c384c22d858909b8"
},
"activo": true,
"fecha_alta": {
"$date": "2026-10-27T18:36:34.431Z"
},
"fecha_baja": null,
"fecha_modificacion": {
"$date": "2026-10-27T18:36:35.018Z"
}
}
{
"_id": {
"$oid": "6ae0ef32c384c22d858909ba"
},
"nombre": "Fugazzeta",
"precio": 9800,
"stock": 14,
"categoria_id": {
"$oid": "6ae0ef32c384c22d858909b8"
},
"activo": false,
"fecha_alta": {
"$date": "2026-10-27T18:36:34.481Z"
},
Alert System Status: All Good
©2026 MongoDB, Inc. Status Terms Privacy Atlas Blog Contact Sales



Prueba de conexión y consultas desde terminal:



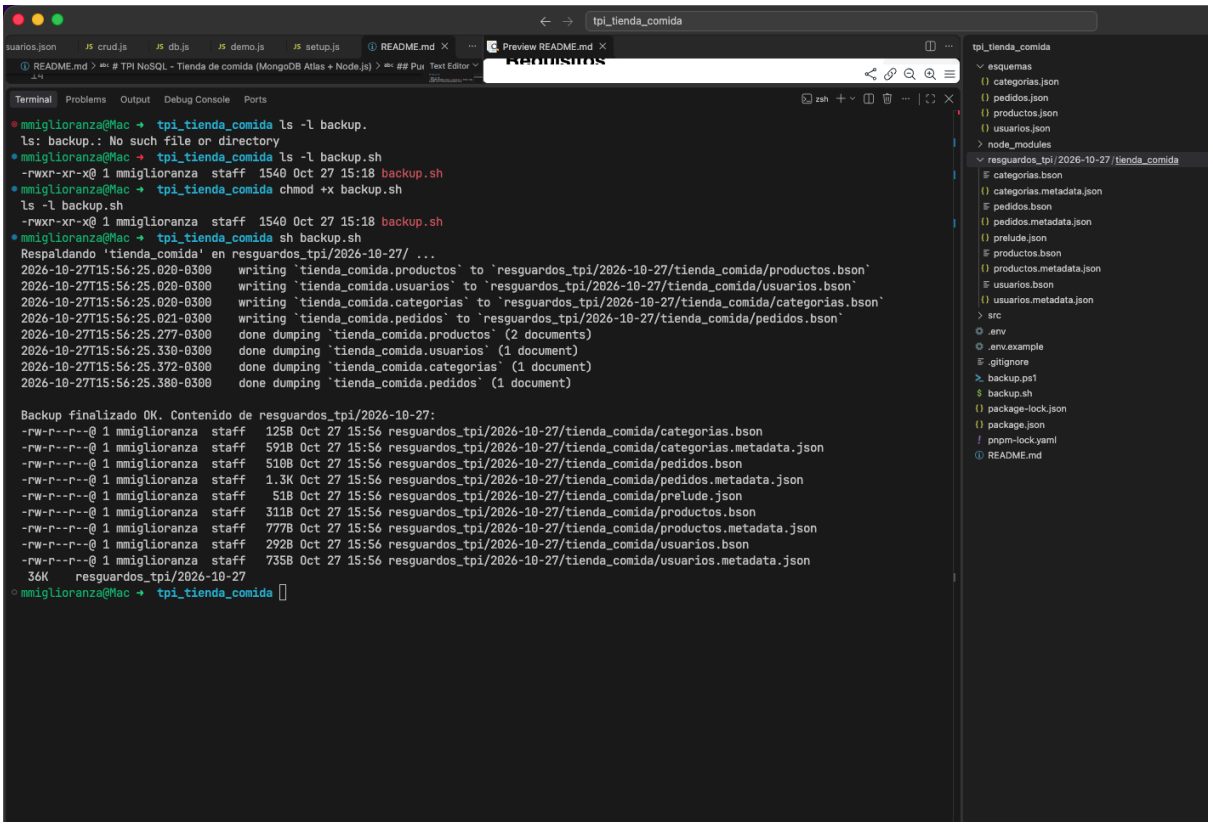
3. Mecanismos de resguardo (backup)

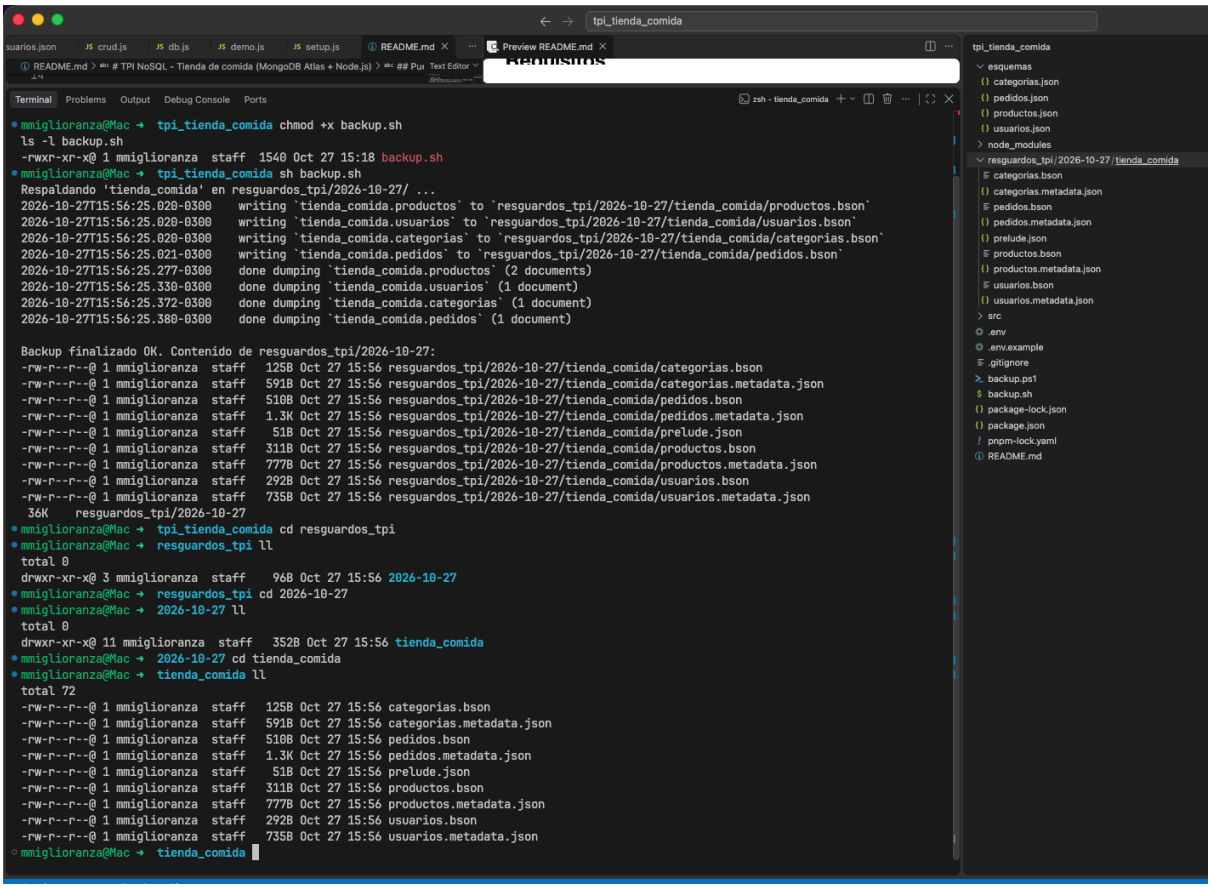
Se utilizó la ayuda de la inteligencia artificial de forma iterativa para poder generar los scripts para sistema UNIX y Windows que realicen el backup pertinente de la base de la base de datos.

Se siguieron estos pasos:

1. Ubicarse en el directorio del proyecto
2. Verificar que mongodump esté instalado
3. Levantar la base de datos (si es que se apunta a una base local)
4. Dar permisos de ejecución al script (en caso de no tenerlos)
5. Ejecutar el script de backup.

Se adjuntan capturas de pantalla de los pasos realizados para verificar el funcionamiento del backup en un entorno local para macOS:





Informe RTO / RPO

Conceptos

- **RPO (Recovery Point Objective):** cantidad máxima de datos que se tolera perder, medida en tiempo. Indica cada cuánto hay que respaldar.
- **RTO (Recovery Time Objective):** tiempo máximo tolerable entre la falla y la vuelta a la operación normal. Indica qué tan rápido se puede restaurar.

Contexto del sistema

La base tienda_comida está alojada en un clúster gratuito M0 de MongoDB Atlas, que no incluye Cloud Backup ni recuperación a un punto en el tiempo. Por eso la única protección propia es el respaldo lógico con mongodump, ejecutado mediante backup.sh (Linux/macOS) y backup.ps1 (Windows). Cada ejecución genera resguardos_tpi/AAAA-MM-DD/tienda_comida/ con un .bson y un .metadata.json por colección.

Datos medidos en la prueba

Dato	Valor
Colecciones respaldadas	4 (usuarios, categorías, productos, pedidos)
Documentos	5
Tamaño del respaldo	36 KB
Duración de mongodump	aprox. 0,4 s (según las marcas de tiempo de la salida)

El volumen es mínimo porque el entorno es de prueba. En una tienda real los números serían mayores, pero mongodump escala de forma lineal, por lo que el análisis se mantiene.

RPO

El RPO es igual al intervalo entre respaldos. Con una ejecución diaria programada (por ejemplo, con cron a las 03:00), el RPO es de **hasta 24 horas**: ante una falla se perderían, como máximo, los pedidos cargados desde el último respaldo. En nuestra prueba el script se ejecutó manualmente, y en ese caso el RPO sería el tiempo transcurrido desde la última ejecución, sin un tope garantizado. Por eso la automatización es lo que permite fijar un RPO definido.

Para una tienda que recibe pedidos durante todo el día, 24 horas es mucho. Recomendamos ejecutar el respaldo cada 4 a 6 horas, lo que lleva el RPO a ese mismo rango, ya que el costo de cada respaldo es insignificante.

RTO

La restauración se realiza con mongorestore --drop sobre la carpeta del día. El RTO total incluye tiempos técnicos y de ejecución de la persona que lo realiza:

Etapa	Tiempo estimado
Detectar la falla y decidir restaurar	5 a 10 min
Ubicar el respaldo correcto y preparar la conexión	2 a 5 min
Ejecutar mongorestore	segundos, con este volumen
Validar los datos restaurados (consultas en Atlas o mongosh)	5 a 10 min
RTO total estimado	15 a 30 min

Se advierten las siguientes limitaciones y mejoras

- mongodump genera una copia lógica, no un snapshot consistente de todo el clúster. Con la carga de este sistema es suficiente.
- Los respaldos se guardan en la misma máquina que ejecuta el script. Conviene copiarlos a otro medio y rotarlos (conservar los últimos N días).
- Un respaldo no probado no es una garantía, por lo que la restauración debería verificarse periódicamente.
- En un clúster pago (M10 o superior), Atlas ofrece snapshots y recuperación a un punto en el tiempo, con un RPO de minutos y un RTO menor. Este sería una buena solución si la tienda obtiene eventualmente los recursos para costearlo.
- La baja lógica complementa el respaldo: ante un borrado por error dentro de la aplicación, el documento sigue en la base y puede recuperarse sin restaurar nada.

4. Conclusión

Reflexión sobre la comunicación Cliente-Servidor

La aplicación Node.js actúa como cliente y el clúster de MongoDB Atlas como servidor. Aunque no es un servidor web con endpoints, la comunicación es real: el driver nativo abre la conexión con la URI de MONGODB_URI y el clúster responde a cada comando con documentos y metadatos como matchedCount y modifiedCount, visibles en la salida de demo.js. Sin perjuicio de ello, para un uso real convendría una API intermedia que evite exponer credenciales, y aplicar mínimo privilegio en lugar del usuario temporal con permisos amplios.

Reflexión sobre la realización del trabajo

La decisión central fue distinguir cuándo embeber y cuándo referenciar. Embebimos el detalle del pedido y las direcciones porque se leen siempre junto a su documento padre, y referenciamos usuarios, categorías y productos por tener ciclo de vida propio. El detalle combina ambas técnicas: guarda el producto_id y copia nombre y precio, de modo que el historial no cambia si el producto se modifica. Aprendimos que esa redundancia es aceptable en NoSQL cuando evita inconsistencias de negocio.

La baja lógica exigió filtrar por activo en cada lectura y actualización, definir cascadas y usar índices únicos parciales. A cambio, no se pierden documentos ni se rompen referencias.

Sobre el resguardo, verificamos que el script genera resguardos_tpi/AAAA-MM-DD con un .bson y un .metadata.json por colección. Entendimos que baja lógica y backup protegen de cosas distintas: errores dentro de la aplicación y fallas de infraestructura. El RPO depende de la frecuencia del respaldo y el RTO del procedimiento de restauración; como el clúster M0 no incluye backups automáticos, mongodump es la protección disponible. Con respaldos diarios automatizados, el sistema tiene un RPO de hasta 24 horas y un RTO estimado de 15 a 30 minutos, adecuado para el contexto académico. Para una operación real, el principal ajuste sería reducir el RPO aumentando la frecuencia de los respaldos y migrar a un clúster con backups continuos.

Por último, usamos inteligencia artificial de forma iterativa. Resultó útil como punto de partida, pero exigió revisar y probar cada resultado: por ejemplo, detectamos que cargar el

.env con source cortaba la URI por el carácter &. Verificar cada paso con evidencia fue tan importante como escribir el código.